MATERIAL DE TRABAJO AUTÓNOMO

APIs REST con Java y Spring Boot

Verbos HTTP, códigos de estado, diseño de endpoints y tu primer controlador REST

Docente: Mgs. Jorge Pantigoso · Lectura previa a la sesión

Logro de la sesión

Al finalizar la sesión, el estudiante **diseña e implementa endpoints REST en Java con Spring Boot**, aplicando correctamente los verbos HTTP y los códigos de estado, y los **prueba con Postman**.

Antes de la clase

Este material es tu punto de partida. En clase **no** repetiremos la teoría desde cero: la usaremos para analizar casos, corregir errores y completar código juntos. Dedica aproximadamente **60 minutos** a estos pasos:

Paso	Actividad	Tiempo
1	Lee las secciones 1 a 5 de este documento.	25 min
2	Clona el repositorio del curso: `git clone https://github.com/jngeor/library.git`	5 min
3	Ejecuta el proyecto y revisa la clase `LibroController` (sección 6).	15 min
4	Prueba los endpoints con Postman usando la colección del repo (sección 7).	10 min
5	Responde las preguntas de autoevaluación (sección 8) y anota tus dudas.	5 min

Requisitos técnicos

- JDK 21.
- Maven 3.9 o superior (IntelliJ IDEA ya lo trae integrado).
- Un IDE: IntelliJ IDEA Community, VS Code con Extension Pack for Java o Eclipse.
- Postman (versión de escritorio o web).
- Git, para clonar el repositorio.
- No necesitas instalar una base de datos: el proyecto usa H2 en memoria.

1. ¿Qué es una API?

Una **API** (*Application Programming Interface*) es un contrato que define cómo un programa puede pedirle información o acciones a otro. Quien la usa no necesita conocer cómo está construido el sistema por dentro: solo necesita saber **qué pedir** y **qué recibirá** como respuesta.

Piensa en un restaurante: tú (el **cliente**) no entras a la cocina (el **servidor**). Le haces tu pedido al mesero (la **API**), que lo lleva a la cocina y te trae el plato (la **respuesta**). El menú sería la documentación de la API: indica qué puedes pedir y cómo.

En el desarrollo web, la forma más común de API es una **API web**: el cliente (una app móvil, un frontend en React, otro sistema) envía peticiones HTTP a una URL, y el servidor responde normalmente con datos en formato **JSON**.

```
Petición (cliente → servidor)          Respuesta (servidor → cliente)
GET /api/libros/1                      HTTP/1.1 200 OK
Host: localhost:8080                   Content-Type: application/json

                                       { "id": 1,
                                         "titulo": "Cien años de soledad",
                                         "autor": "Gabriel García Márquez" }
```

2. ¿Qué significa REST?

REST (*Representational State Transfer*) es un estilo de arquitectura propuesto por Roy Fielding en su tesis doctoral (Fielding, 2000). No es un lenguaje ni una librería: es un conjunto de principios. Una API que los sigue se llama **API REST** o **RESTful**. Los principios más importantes para esta sesión son:

Principio	Qué significa en la práctica
Recursos identificados por URI	Todo lo que la API expone es un **recurso** (libros, usuarios, pedidos) con una dirección única: `/api/libros/1`.
Interfaz uniforme	Se usan los **verbos HTTP** estándar para operar sobre los recursos, en lugar de inventar acciones en la URL.
Sin estado (stateless)	Cada petición contiene toda la información necesaria. El servidor no recuerda peticiones anteriores del cliente.
Cliente-servidor	El cliente y el servidor evolucionan por separado; solo comparten el contrato de la API.
Representaciones	El recurso se transfiere en un formato (normalmente JSON); el cliente nunca accede directamente a la base de datos.

3. Verbos HTTP

El **verbo** (o método) indica **qué acción** se quiere hacer sobre el recurso. La URL indica **sobre qué**. Esta separación es la base de un buen diseño REST (Fielding et al., 2022).

Verbo	Acción	Ejemplo	¿Idempotente?
GET	Obtener uno o varios recursos. No modifica nada.	`GET /api/libros` `GET /api/libros/1`	Sí
POST	Crear un recurso nuevo.	`POST /api/libros`	No
PUT	Reemplazar un recurso completo.	`PUT /api/libros/1`	Sí
PATCH	Modificar parcialmente un recurso.	`PATCH /api/libros/1`	No necesariamente
DELETE	Eliminar un recurso.	`DELETE /api/libros/1`	Sí

> **Concepto clave: idempotencia**
>
> Una operación es **idempotente** si ejecutarla una vez o varias veces deja el servidor en el mismo estado. Si envías tres veces `DELETE /api/libros/1`, el libro queda eliminado igual que con una sola petición. En cambio, tres `POST /api/libros` crearían **tres libros distintos**.

4. Códigos de estado HTTP

Cada respuesta incluye un **código de estado** de tres dígitos que le dice al cliente qué pasó con su petición. El primer dígito indica la familia:

Familia	Significado	Responsable
2xx	Éxito: la petición se procesó correctamente.	—
4xx	Error del cliente: la petición está mal formada o pide algo que no existe.	El cliente
5xx	Error del servidor: algo falló al procesar una petición válida.	El servidor

Los códigos que usarás con más frecuencia:

Código	Nombre	Cuándo se usa
200	OK	GET, PUT o PATCH exitosos que devuelven datos.
201	Created	POST exitoso: se creó un recurso nuevo.
204	No Content	Operación exitosa sin cuerpo de respuesta (frecuente en DELETE).
400	Bad Request	Datos inválidos o incompletos (por ejemplo, falta un campo obligatorio).
404	Not Found	El recurso solicitado no existe.
500	Internal Server Error	Error inesperado en el servidor.

Error frecuente

Responder siempre **200 OK** y poner el error dentro del JSON (por ejemplo, `{"error": "no existe"}`). Esto confunde al cliente: el código de estado debe reflejar lo que realmente pasó.

5. Buenas prácticas para diseñar endpoints

Un **endpoint** es la combinación de un verbo HTTP y una ruta (por ejemplo, `GET /api/libros/1`). Estas reglas hacen que una API sea predecible y fácil de usar (Massé, 2011):

- **Usa sustantivos, no verbos, en la ruta.** La acción ya la indica el verbo HTTP.
- **Usa el plural para las colecciones:** `/libros`, `/usuarios`.
- **Identifica un elemento con su id en la ruta:** `/libros/5`.
- **Expresa relaciones con jerarquía:** `/autores/3/libros` (los libros del autor 3).
- **Usa minúsculas** y guiones para separar palabras: `/prestamos-activos`.
- **Usa parámetros de consulta para filtrar u ordenar:** `/libros?autor=Vargas`.

Intención	Evita	Prefiere
Listar todos los libros	`GET /obtenerLibros`	`GET /api/libros`
Crear un libro	`POST /crearLibro`	`POST /api/libros`
Ver los préstamos del usuario 7	`GET /prestamosDeUsuario?id=7`	`GET /api/usuarios/7/prestamos`

En clase analizaremos más casos como estos, así que fíjate bien en el patrón.

6. Tu primera API REST con Spring Boot

Spring Boot es un framework de Java que permite crear APIs REST con muy poca configuración (Spring, s. f.; Walls, 2022). Usa **anotaciones** para indicar qué clase atiende peticiones y qué método responde a cada endpoint:

Anotación	Para qué sirve
`@RestController`	Marca la clase como controlador REST: sus métodos devuelven datos (JSON), no vistas HTML.
`@RequestMapping`	Define la ruta base de todos los endpoints de la clase, por ejemplo `/api/libros`.
`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`	Asocian un método a un verbo HTTP.
`@PathVariable`	Toma un valor de la ruta, como el `{id}` en `/api/libros/{id}`.
`@RequestBody`	Convierte el JSON del cuerpo de la petición en un objeto Java.
`@Valid`	Valida los datos recibidos; si son incorrectos, Spring responde **400 Bad Request**.
`ResponseEntity`	Permite controlar el código de estado y el cuerpo de la respuesta.
`@Entity`, `@Table`	Indican que una clase representa una tabla de la base de datos.

Cómo está organizado el proyecto

El proyecto usa una sola tabla, `libros`, en una base de datos **H2 en memoria**, y se organiza en tres capas. Cada una tiene una única responsabilidad:

Clase	Capa	Responsabilidad
`Libro`	Modelo	Representa un registro de la tabla `libros`.
`LibroRepository`	Acceso a datos	Lee y guarda libros en la tabla.
`LibroController`	Controlador	Recibe las peticiones HTTP y decide qué responder.

```
Petición:   Postman → LibroController → LibroRepository → tabla "libros"
Respuesta:  Postman ← JSON + código de estado ← LibroController
```

El modelo: la tabla libros

La clase `Libro` es una **entidad**: Spring crea automáticamente la tabla a partir de sus atributos. Las validaciones `@NotBlank` impiden guardar un libro sin título o sin autor.

```
@Entity
@Table(name = "libros")
public class Libro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;                 // lo genera la base de datos

    @NotBlank(message = "El título es obligatorio")
    private String titulo;

    @NotBlank(message = "El autor es obligatorio")
    private String autor;

    // constructores, getters y setters
}
```

El repositorio

Basta con declarar una interfaz que extienda `JpaRepository`. Spring Data genera por nosotros los métodos `findAll`, `findById`, `save`, `existsById` y `deleteById`:

```
public interface LibroRepository extends JpaRepository<Libro, Long> {
}
```

El controlador

Estos son los dos métodos GET de la clase `LibroController`. El controlador recibe el repositorio por su constructor (**inyección de dependencias**) y lo usa para consultar la tabla:

```
@RestController
@RequestMapping("/api/libros")
public class LibroController {

    private final LibroRepository repository;

    public LibroController(LibroRepository repository) {
        this.repository = repository;
    }

    // GET /api/libros  -> 200 OK con la lista completa
    @GetMapping
    public List<Libro> listar() {
        return repository.findAll();
    }

    // GET /api/libros/{id}  -> 200 OK si existe, 404 Not Found si no
    @GetMapping("/{id}")
    public ResponseEntity<Libro> obtener(@PathVariable Long id) {
        return repository.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    // En el repositorio encontrarás también:
    // crear (POST), actualizar (PUT) y eliminar (DELETE)
}
```

Analiza antes de la clase

- ¿Por qué `listar()` devuelve directamente una lista y `obtener()` devuelve un `ResponseEntity`?
- ¿Qué pasaría si `obtener()` devolviera `null` cuando el libro no existe, en lugar de un 404?
- `LibroRepository` no tiene ningún método escrito. ¿Cómo es posible que funcione?
- Revisa en el repo los métodos POST, PUT y DELETE: ¿qué código de estado devuelve cada uno y por qué?

7. Probar la API con Postman

Postman es una herramienta para enviar peticiones HTTP sin necesidad de construir un frontend. Con el proyecto en ejecución:

Paso	Qué hacer
1	Ejecuta la aplicación desde tu IDE. Por defecto se levanta en `http://localhost:8080`.
2	En Postman, usa `Import` y selecciona el archivo `postman/API_Libros.postman_collection.json` del repositorio. Trae un caso de éxito y uno de error para cada verbo.
3	Envía `GET http://localhost:8080/api/libros`. Deberías recibir **200 OK** y la lista de libros.
4	Envía `GET http://localhost:8080/api/libros/1` y luego con el id `99`. Compara los códigos de estado.
5	Observa en cada respuesta: el **código de estado**, el **cuerpo JSON** y el **tiempo de respuesta**.
6	Abre `http://localhost:8080/h2-console` (JDBC URL: `jdbc:h2:mem:biblioteca`, usuario `sa`, sin contraseña) y ejecuta `SELECT * FROM libros;`. Repite la consulta después de un POST o un DELETE y observa el cambio en la tabla.

Los datos se reinician cada vez que vuelves a ejecutar la aplicación, así que puedes experimentar sin miedo.

8. Autoevaluación

Responde sin mirar el material y luego verifica tus respuestas. Si fallas alguna, repasa la sección indicada. Anota tus dudas para la clase.

#	Pregunta	Repasa
1	Un cliente envía un POST con un libro sin título. ¿Qué código de estado debería devolver la API?	Sección 4
2	¿Por qué `DELETE` es idempotente y `POST` no lo es?	Sección 3
3	Corrige este endpoint: `GET /api/borrarLibro?id=3`	Sección 5
4	¿Qué diferencia hay entre `@PathVariable` y `@RequestBody`?	Sección 6
5	Diseña el endpoint para obtener todos los libros de la autora con id 4.	Sección 5

Para pensar e indagar

Tal como está, cualquier persona que conozca la URL podría eliminar libros de nuestra API. ¿Cómo podríamos evitarlo? Investiga brevemente qué es la **autenticación** en una API REST y tráelo a la clase.

Glosario

Término	Definición
Endpoint	Combinación de verbo HTTP y ruta a la que responde la API.
Recurso	Entidad que la API expone y manipula (un libro, un usuario).
JSON	Formato de texto ligero para intercambiar datos, basado en pares clave-valor.
Idempotencia	Propiedad de una operación que produce el mismo resultado si se repite.
Controlador	Clase que recibe las peticiones HTTP y decide qué responder.

Referencias

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://doi.org/10.17487/RFC9110

Massé, M. (2011). *REST API design rulebook*. O'Reilly Media.

Spring. (s. f.). *Building a RESTful web service*. https://spring.io/guides/gs/rest-service

Walls, C. (2022). *Spring in action* (6.ª ed.). Manning.